



VNIVERSITAT
ID VALÈNCIA



ISP · Image & Signal Processing

Índice del curso.
Introducción a conda,
mamba, venv

Oscar José Pellicer Valero

Oscar.Pellicer@uv.es

Departament d'Enginyeria Electrònica
Escola Tècnica Superior d'Enginyeria
Universitat de València



Prompt: A toolbox for data scientists, data all over the place

Índice del curso: días 1 y 2

Estos serán los materiales principales que veréis cada día. Tendréis una combinación de **slides en PDF** para la parte conceptual y **notebooks en `.ipynb`** para la parte práctica.

Día 1

- **Slides:** `1_setup.pdf`
- **Notebook:** `day1_intro_nb_opencv.ipynb` y `day1_nb_opencv.ipynb`
- **Aprenderéis:** a preparar el entorno, cargar imágenes, hacer preprocesado básico con `NumPy` y `OpenCV` y construir una primera regla heurística para clasificar imágenes correctas e incorrectas.

Día 2

- **Slides:** `3_cnns.pdf` y `4_evaluation.pdf`
- **Notebook:** `day2_intro_nb_cnns.ipynb` y `day2_nb_cnns.ipynb`
- **Aprenderéis:** a entrenar una CNN con `PyTorch`, evaluar el modelo con matrices de confusión, y visualizar en qué zonas de la imagen se fija la red.

Índice del curso: días 3 y 4

Día 3

- **Slides:** `5_fms.pdf`
- **Notebook:** `day3_intro_nb_llm_classification.ipynb` y `day3_nb_llm_classification.ipynb`
- **Aprenderéis:** a usar modelos fundacionales para clasificación visual *zero-shot* (Qwen, Gemini y), a probar prompting multimodal y a entender cómo reutilizar embeddings visuales para tareas médicas.

Día 4

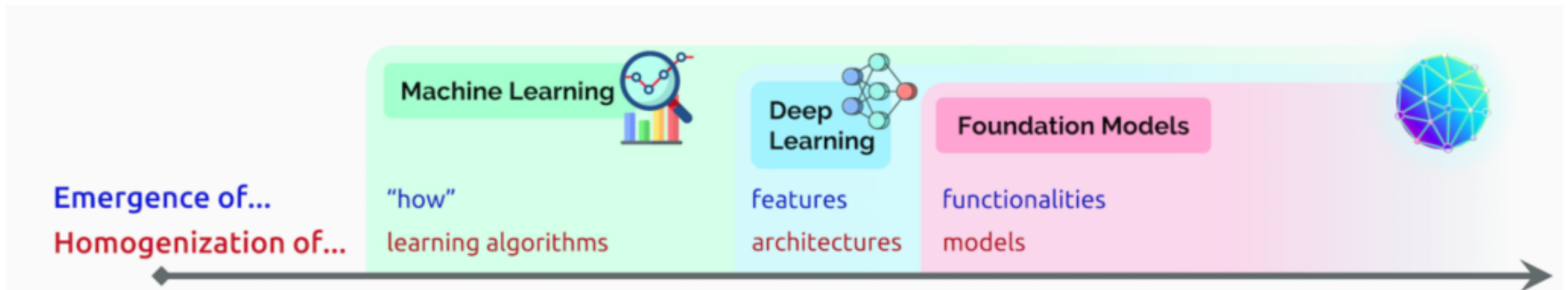
- **Slides:** no habrá presentación
- **Notebook:** `day4_intro_nb_streamlit.ipynb` y `day4_nb_streamlit.ipynb`
- **Aprenderéis:** a convertir el modelo en una aplicación con Streamlit, cargar imágenes, mostrar predicciones y preparar una base para inferencia con webcam y feedback visual.

Objetivo final

Al final del curso habréis recorrido un flujo completo: **datos** → **modelo** → **evaluación** → **aplicación interactiva**.

El curso en contexto: la revolución de los modelos fundacionales

Este curso representará un paralelo con la evolución de las técnicas clásicas de visión por computadora → CNNs → Modelos Fundacionales, permitiéndonos entender el lugar de cada uno de estos paradigmas en el panorama actual, y si lo clásico sigue siendo útil o si los modelos fundacionales lo han dejado obsoleto.



Bommasani, Rishi, et al. "On the opportunities and risks of foundation models." arXiv:2108.07258 (2021).

Desinstalando Anaconda

Si ya tienes instalada una versión de Anaconda, para evitar conflictos, es muy recomendable desinstalarla por completo antes de empezar:

Windows:

1. **Usar el desinstalador:** Busca "Agregar o quitar programas", encuentra Anaconda en la lista y haz clic en "Desinstalar". En la carpeta de instalación de Anaconda (ej. `C:\Users\username\Anaconda3`) suele haber un `Uninstall-Anaconda3.exe` .
2. **Limpieza manual:** Después de desinstalar, borra las carpetas residuales como `.conda` y `.continuum` en tu directorio de usuario (`C:\Users\username`).

macOS / Linux:

1. **Eliminar la carpeta de instalación:** Abre una terminal y elimina el directorio de Anaconda: `rm -rf ~/anaconda3` (la ruta puede variar, p.ej., `~/opt/anaconda3`).
2. **Limpiar archivos de configuración:** Elimina las carpetas y archivos de configuración ocultos en tu directorio de inicio: `rm -rf ~/.condarc ~/.conda ~/.continuum` .
3. **Editar el perfil del shell:** Abre tu archivo de configuración del shell (p.ej., `~/.bashrc` , `~/.zshrc`) y elimina la línea que añade Anaconda a tu `PATH` .

Instalando Miniconda

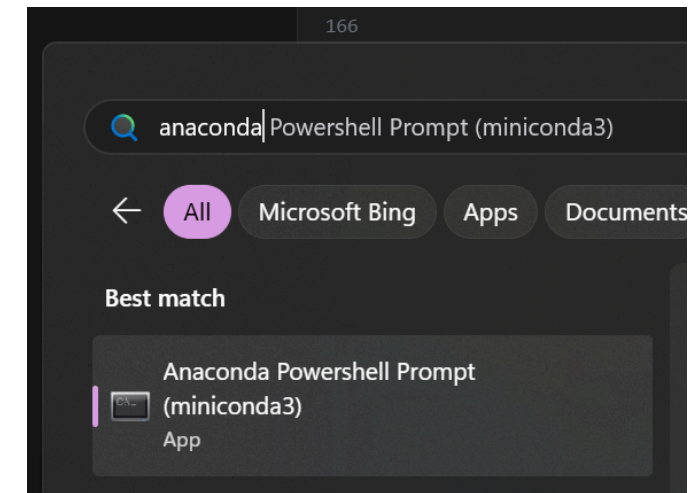


Miniconda es una versión mínima de Anaconda que incluye solo Python, `conda` y un pequeño número de paquetes. Esto nos da un mayor control sobre nuestros entornos.

Para instalar **Miniconda**, accede a la [página de descarga](#), haz click en `Get Started`, escoge la opción adecuada para tu sistema operativo, descarga el instalador y instálalo (deja todas las opciones por defecto).

Una vez instalado, para empezar a trabajar con Anaconda:

- En Windows, abre el Anaconda Prompt desde el menú de inicio (ver imagen de la derecha →)
- En macOS, abre la terminal haciendo `Command + Space` y buscando `Terminal`
- En Linux, abre la terminal haciendo `Ctrl + Alt + T`



Instalando Visual Studio Code

Visual Studio Code (VS Code) es un editor de código ligero y potente que usaremos como nuestra herramienta principal para trabajar con Python y Jupyter Notebooks.



1. Descarga e instala VS Code:

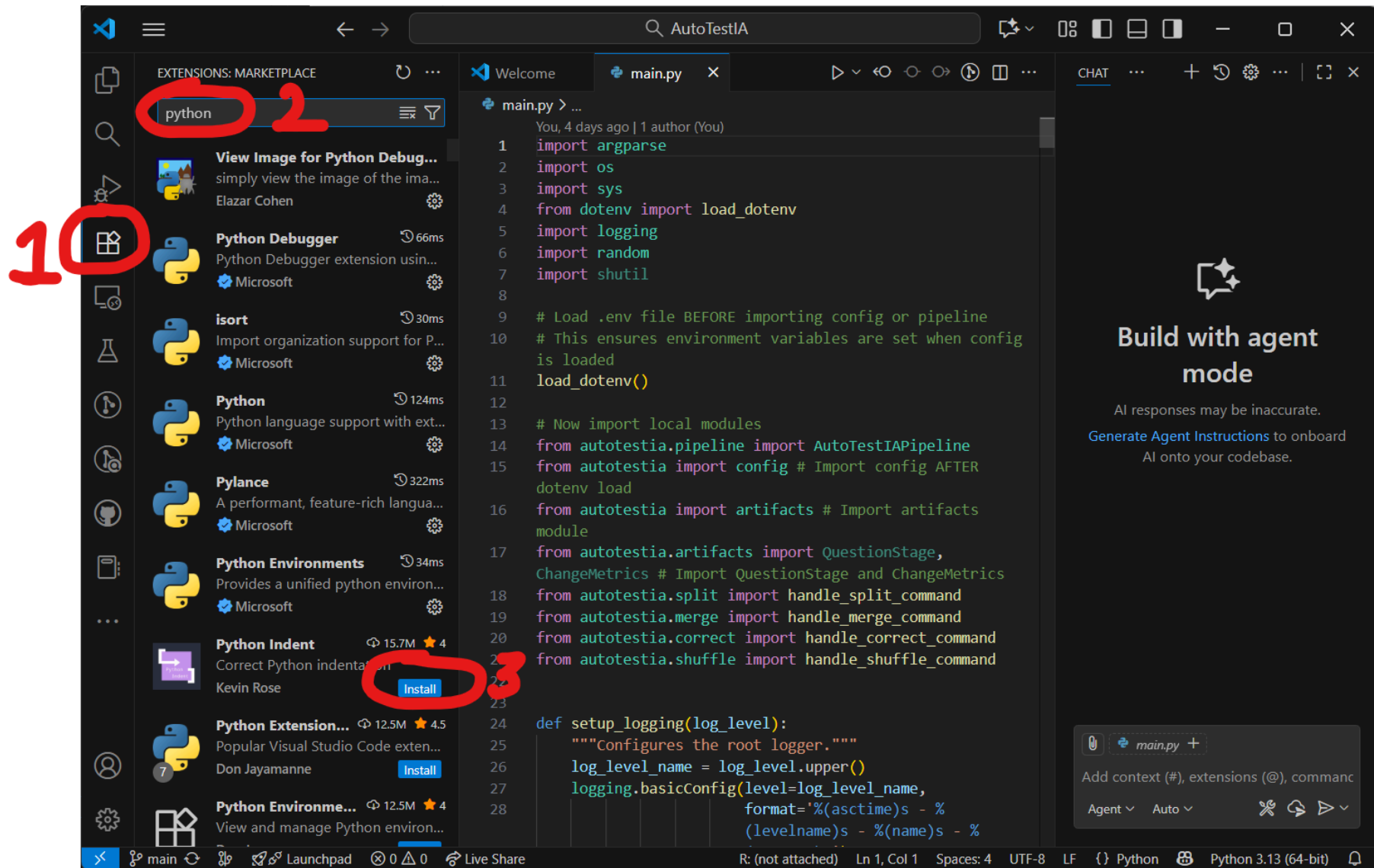
Ve a la [web oficial de VS Code](#) y descarga el instalador para tu sistema operativo. Sigue las instrucciones de instalación.

2. Instala las extensiones esenciales:

Una vez abierto VS Code, ve a la pestaña de Extensiones (el icono de los cubos en la barra lateral) y busca e instala las siguientes extensiones del desarrollador Microsoft:

- **Python:** Soporte para el lenguaje Python (linting, debugging, etc.).
- **Jupyter:** Soporte para trabajar con Jupyter Notebooks.

Notar que al instalar las extensiones anteriores, VSCode instalará automáticamente muchas otras asociadas. En la siguiente diapositiva se muestra cómo instalar las extensiones paso a paso ↓



Usando Jupyter Notebooks en VS Code

1. Abre la carpeta de tu proyecto en VS Code

Puedes usar `File > Open Folder...` o arrastrar la carpeta directamente a la ventana de VS Code.

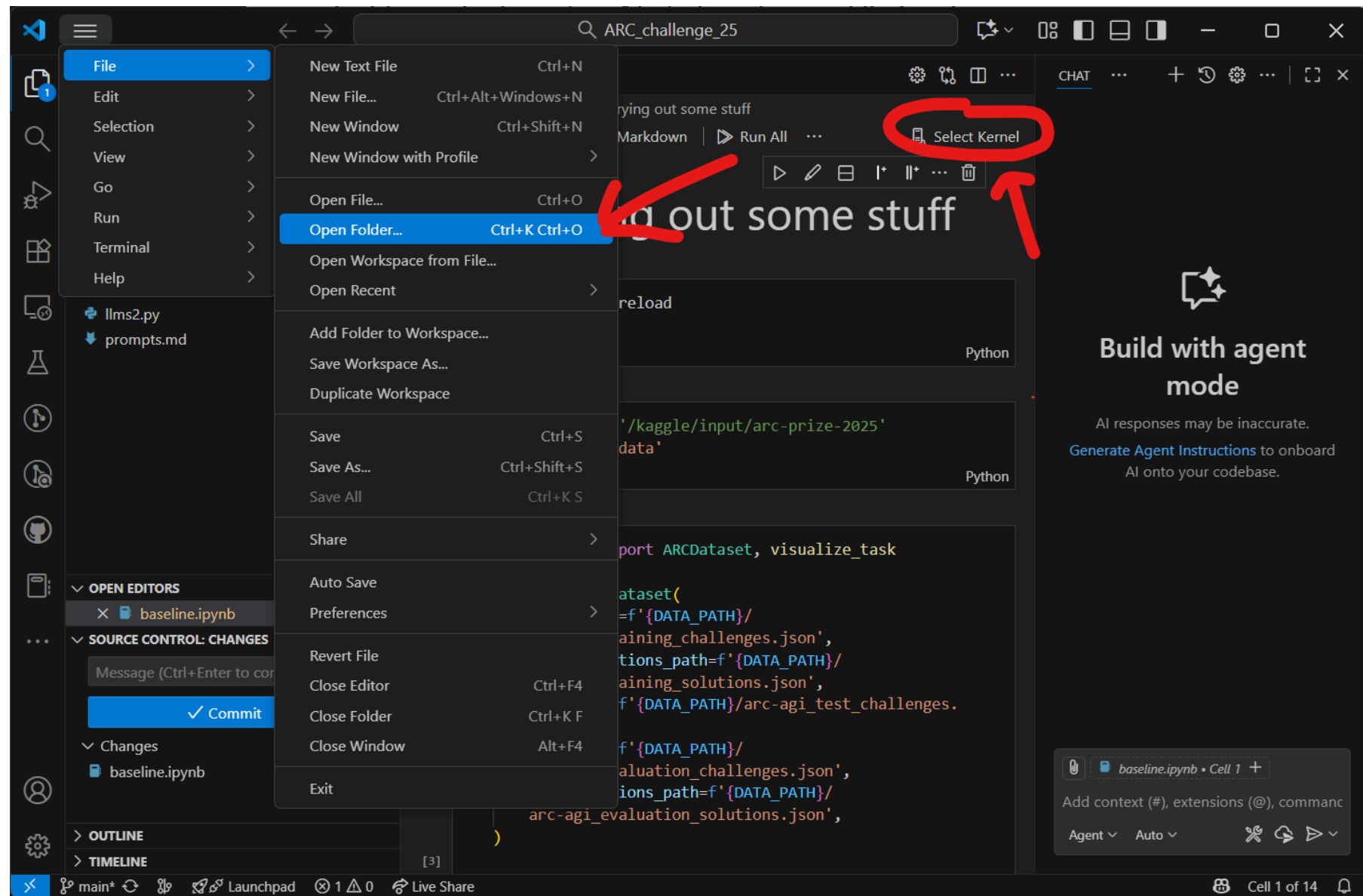
2. Crea o abre un Jupyter Notebook

- Para crear uno nuevo, puedes usar el atajo `Ctrl+Shift+P` para abrir la paleta de comandos, escribir `Jupyter: Create New Jupyter Notebook` y presionar Enter.
- Si ya tienes un archivo `.ipynb`, simplemente haz doble clic sobre él en el explorador de archivos de VS Code.

3. Selecciona el kernel correcto

En la esquina superior derecha del notebook, verás un botón para seleccionar el `kernel`. Haz clic en él y elige tu entorno de conda `ds`. Si no aparece, puede que tengas que buscarlo haciendo clic en `Select Another Kernel...` y navegando hasta el ejecutable de Python de tu entorno.

Mira la imagen en la siguiente slide como referencia ↓



Accediendo a la versión Pro de GitHub Copilot

GitHub Copilot es un asistente de programación basado en IA que se integra en VS Code y te ayuda a escribir código más rápido. Como estudiantes, podéis acceder a Copilot Pro de forma gratuita solicitando a Github acceso los *Education Benefits*.



1. Regístrate en GitHub y añade tu correo académico:

Si no tienes una cuenta, [regístrate en GitHub](#) usando tu correo electrónico personal (gmail, yahoo, etc.). A continuación, añade tu correo académico a tu cuenta desde la sección [Emails](#) en *Settings* (ver siguiente slide ↓)

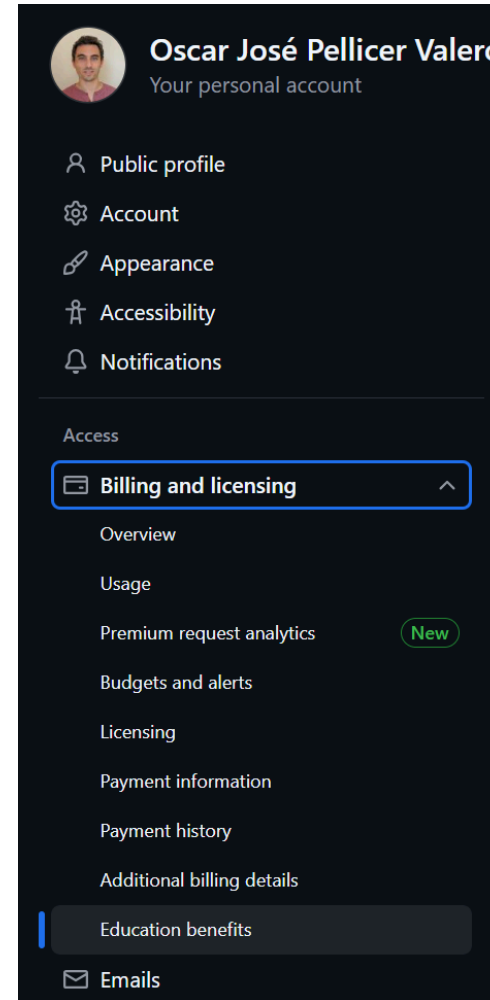
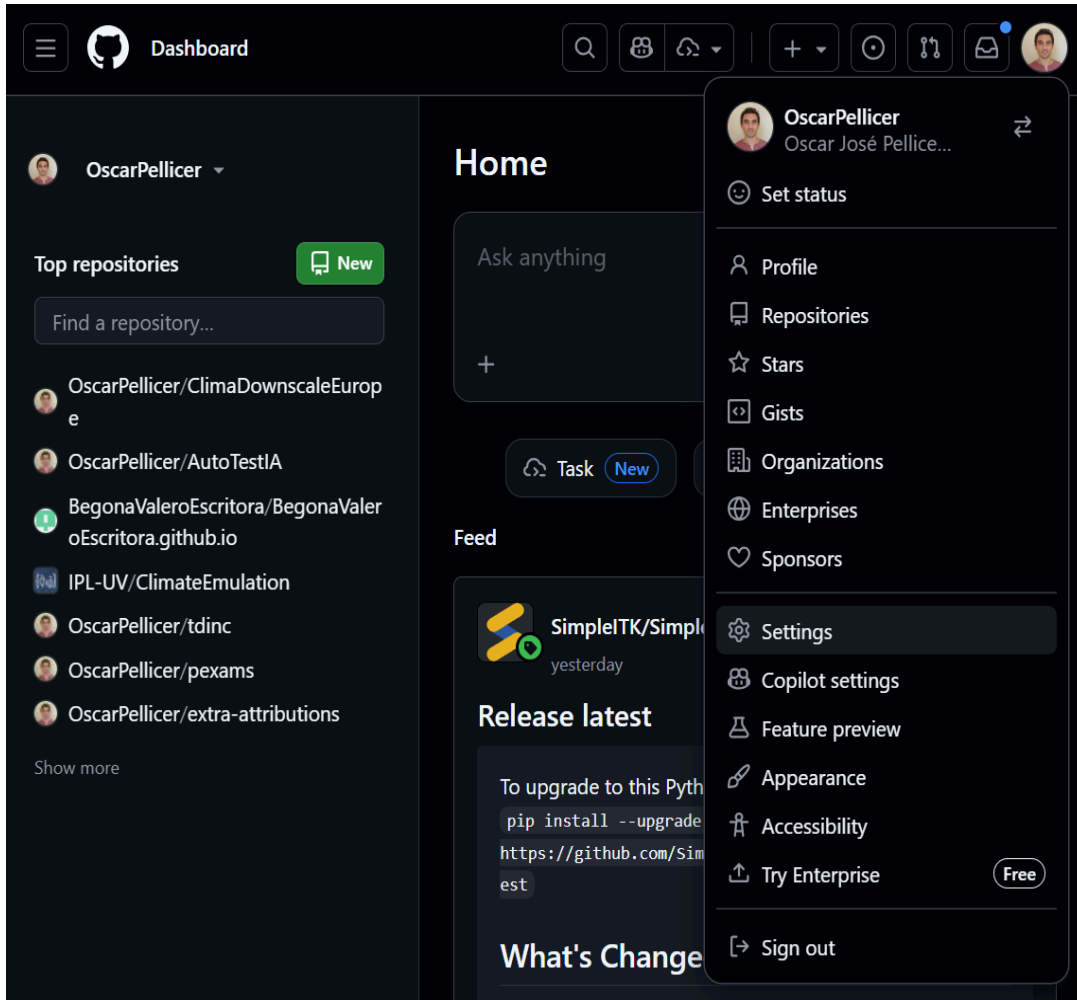
2. Solicita acceso a los *Education Benefits*:

Ve a [esta página](#) y sigue los pasos para solicitar acceso a los *Education Benefits*. Una vez aprobados, podrás acceder a Copilot Pro de forma gratuita. Nota que para ello necesitarás:

- Subir una foto de tu carnet de estudiante (puedes usar la imagen que aparece en la intranet)
- Tener activado la [autenticación en dos factores](#) en tu cuenta de GitHub (puedes usar una aplicación de autenticación como Microsoft Authenticator)
- Tener tu nombre y apellidos completos en [tu perfil de GitHub](#)

3. Instala la extensión en VS Code:

Ve a la pestaña de Extensiones en VS Code, busca `Github` y `GitHub Copilot` e instálalas.



Creando un entorno para Ciencia de Datos

Vamos a crear un entorno llamado `nlp` con las librerías esenciales.

1. Abre tu terminal (macOS/Linux) o Anaconda Prompt (Windows).
2. Crea el entorno instalando los paquetes directamente:

```
conda create -n nlp python=3.11 numpy pandas matplotlib scikit-learn ipykernel
```

- `conda create -n nlp` : Crea un nuevo entorno llamado `nlp` .
- `python=3.11` : Especifica la versión de Python.
- `numpy pandas ...` : Lista de paquetes a instalar.

3. Activa el nuevo entorno:

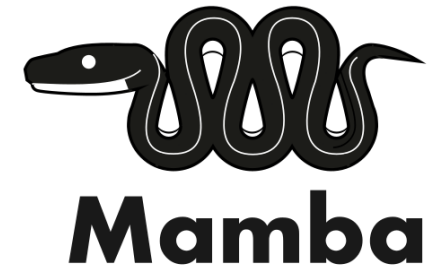
```
conda activate nlp
```

Tu terminal cambiará para mostrar `(nlp)` , indicando que el entorno está activo.

Potenciando `conda` con `mamba`

Mamba es una reimplementación de `conda` en C++ que es mucho más rápida, especialmente a la hora de resolver dependencias complejas.

Una vez instalado Miniconda, abre el **Anaconda Prompt (Windows)** o tu **terminal (macOS/Linux)** y ejecuta:



```
conda install -n base -c conda-forge mamba
```

A partir de ahora, para la mayoría de los comandos, puedes usar `mamba` en lugar de `conda` para una experiencia mucho más fluida y rápida.

Comandos básicos de `conda` / `mamba`

Gestión de entornos:

- Crear un entorno:

```
conda create --name mi_entorno python=3.10
```

- Activar un entorno:

```
conda activate mi_entorno
```

- Desactivar el entorno actual:

```
conda deactivate
```

- Listar todos los entornos:

```
conda env list
```

- Eliminar un entorno:

```
conda env remove --name mi_entorno
```

Gestión de paquetes:

- Instalar paquetes:

```
mamba install numpy pandas scikit-learn
```

- Instalar desde `conda-forge` (más paquetes):

```
mamba install -c conda-forge nombre_paquete
```

- Listar paquetes instalados en el entorno activo:

```
mamba list
```

- Eliminar un paquete:

```
mamba remove nombre_paquete
```

- Exportar un entorno a un archivo y crearlo desde él:

```
mamba env export > entorno.yml  
mamba env create -f entorno.yml
```

Alternativa a VS Code: Jupyter Lab

Si prefieres la interfaz clásica de Jupyter Lab en el navegador, puedes seguir estos pasos.

1. **Instala Jupyter Lab en el entorno `base`** : Jupyter Lab es una interfaz web interactiva. Lo instalaremos en el entorno `base` para que actúe como nuestro "lanzador" principal.

```
mamba install -n base -c conda-forge jupyterlab
```

2. **Conecta tu entorno `nlp`** : Por defecto, Jupyter Lab solo "ve" el Python del entorno desde el que se lanza. Para que pueda usar nuestro entorno `nlp` , necesitamos instalar `ipykernel` en él y registrarlo.

- **Activa el entorno `nlp`** : `conda activate nlp`
- **Instala `ipykernel`** : `mamba install ipykernel`
- **Registra el entorno:** `python -m ipykernel install --user --name ds --display-name ds`

3. **Lanza Jupyter Lab:**

Asegúrate de estar en el entorno `base` (`conda deactivate` si estás en otro) y ejecuta: `jupyter lab`

Esto abrirá Jupyter Lab en tu navegador, y ahora verás "ds" como una opción para crear nuevos notebooks.

Alternativa a `conda` / `mamba`: `venv`

Python incluye su propio módulo para crear entornos virtuales ligeros, llamado `venv`.

Creación y activación:

1. Navega a la carpeta de tu proyecto.
2. Crea el entorno:

```
python -m venv .venv
```

(Se creará una carpeta `.venv` en tu directorio actual)

3. Activa el entorno:

- **Windows:**

```
.venv\Scripts\activate
```

- **macOS / Linux:**

```
source .venv/bin/activate
```

Gestión de paquetes:

- `venv` no gestiona paquetes. Se usa `pip`, el instalador de paquetes de Python.

- **Instalar paquetes:**

```
pip install numpy pandas
```

- **Guardar dependencias:**

```
pip freeze > requirements.txt
```

- **Instalar desde un archivo:**

```
pip install -r requirements.txt
```

conda / mamba vs. venv : Diferencias clave

conda / mamba

- **Gestor de paquetes Y de entornos.**
- **Independiente del lenguaje:** Puede gestionar paquetes y dependencias de otros lenguajes (R, C++, etc.), no solo Python.
- **Gestión de binarios:** Instala paquetes pre-compilados (binarios), lo que puede ser más rápido y evitar problemas de compilación, especialmente en Windows.
- **Gestión de Python:** Puede instalar diferentes versiones de Python en diferentes entornos.
- **Centralizado:** Los entornos se guardan en una carpeta central de Miniconda, no junto al proyecto.

venv

- **Solo gestor de entornos.** Para paquetes, se usa `pip`.
- **Específico de Python:** Solo gestiona paquetes de Python.
- **Instala desde "wheels" o fuentes:** `pip` instala paquetes binarios (wheels) o compila desde el código fuente si no hay un wheel disponible para tu sistema.
- **Usa el Python con el que se creó:** El entorno usa la versión de Python del sistema que se usó para crearlo.
- **Descentralizado:** El entorno se crea típicamente dentro de la carpeta del proyecto.

Recomendación: Para este curso, usaremos `conda` + `mamba` por su robustez en el ecosistema de ciencia de datos.